

Yu-Chang Shih

shih.yuc@northeastern.edu • (+1) 408-210-4509 • [LinkedIn](#) • [Github](#)

Automated Testing Library for Turn-Based and Zero-Sum Games

Overview

Developed a versatile automated testing library for turn-based and zero-sum games, capable of validating game logic, verifying frontend and backend data, and detecting rare bugs that conventional API testing could not catch. The library leverages computer vision, automated input simulation, and modular pipelines to accelerate testing cycles and reduce manual effort.

Technical Challenges

- **Manual calculation bottleneck**
Traditional verification of in-game numeric data was error-prone and extremely time-consuming, especially for test cases related to numbers and flow checks.
- **Low-Frequency Scenarios**
API-forced triggers can save time but fail to guarantee the reliability of naturally occurring, long-running processes.
- **Mechanical Gaps Impacting Gameplay Integrity**
Detecting "sure-win" strategies is difficult via standard play testing, yet their existence subtly erodes the intended challenge and long-term engagement.

Approach

- **Screen-Based Automation and Object Detection**
 - Used OpenCV and Fast R-CNN to detect in-game objects and numeric data directly from screenshots.
 - Automated mouse actions using PyAutoGUI, with action space defined as screen coordinates for click events.
 - Enabled users to define reward functions for AI-driven gameplay, allowing reinforcement learning integration.
- **Scalable Library Design and Team Enablement**
 - Designed a modular testing framework with parallel execution and asynchronous validation.
 - Trained teammates to use the library by modularizing the system and assigning clear ownership, enabling independent expansion of test coverage across new games and additional data extraction.
- **Rare Bug Detection and Verification**
 - Highlighted low-frequency test cases in HTML reports and detected potential game-breaking strategies through AI action.
 - Verified numeric data automatically, reducing manual calculation errors and human effort.

Results

- Detected rare bugs missed by standard API tests, including floating-point precision discrepancies.
- Reduced team workload by 30% while improving test coverage and establishing reliable regression testing through automation.
- Created a framework that is extensible to AI-driven gameplay, allowing future reinforcement learning models to train directly on screen inputs and automatically improve performance.